

CKA Intensive

Day 4 — Cluster Lifecycle, CRDs, Troubleshooting, Mock Exam

Target: **Kubernetes v1.32**

The exam-prep day. We build a cluster, recover from disasters, learn the troubleshooting playbook, then sit a mock.

Today

1. **Morning quiz** (10 min, oral)
2. kubeadm — bootstrap, join, upgrade, HA topologies
3. **TLS cert management** — what expires, when, how to rotate
4. **Lab 7** — build a cluster on multipass/Hetzner from scratch
5. etcd backup & restore
6. **Lab 8** — etcd
7. **CRDs and operators** overview
8. **Lab 8b** — install an operator, observe reconciliation
9. Troubleshooting playbook + logging + metrics
10. **Lab 9** — broken cluster (random scenario)
11. **Mock Exam — 75 min, no notes, no Google, no AI**
12. Mock review, exam-day strategy, wrap-up

kubeadm — what it does, what it doesn't

Does:

- Generates the PKI (`/etc/kubernetes/pki/`)
- Writes static pod manifests for apiserver, scheduler, controller-manager, etcd (`/etc/kubernetes/manifests/`)
- Bootstraps the kubelet via the kubelet-config ConfigMap
- Issues join tokens so workers can register
- Manages upgrades

Does NOT:

- Install a CNI (you do)
- Install monitoring/logging
- Install a CSI/cloud-controller
- Manage cluster lifecycle beyond install/upgrade (no scaling, no recovery)

Pre-flight (every node, every time)

```
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab          # persist

sudo modprobe overlay
sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
```

Then install: **containerd** + kubelet + kubeadm + kubectl, all matching the target version.

containerd — the runtime

```
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
# critical: set SystemdCgroup = true so the cgroup driver matches kubelet
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
```

If you forget `SystemdCgroup = true`, kubelet and containerd will fight over cgroups → flapping nodes. This is the #1 kubeadm install bug.

Install kubelet, kubeadm, kubectl (v1.32)

```
sudo mkdir -p -m 755 /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /" | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl          # pin
```

The repo URL is **versioned** per minor. To upgrade to 1.33, change the URL.

kubeadm init (control plane)

```
sudo kubeadm init \
  --pod-network-cidr=192.168.0.0/16 \           # Calico default
  --apiserver-advertise-address=$(hostname -i) \
  --upload-certs

# save the join command from the output, e.g.:
# kubeadm join 10.0.0.5:6443 --token <t> \
#   --discovery-token-ca-cert-hash sha256:<h>

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

You can regenerate the join command anytime:

```
kubeadm token create --print-join-command
```

Install a CNI

`k get nodes` shows `NotReady` until a CNI is in place. Calico:

```
k apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/tigera-operator.yaml
k apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/custom-resources.yaml

k get pods -A -w          # wait for calico-system to come up
k get nodes               # control plane → Ready
```

The pod CIDR in `custom-resources.yaml` (default `192.168.0.0/16`) must match `--pod-network-cidr` from `kubeadm init`.

Join workers

On each worker (after pre-flight + containerd + kubeadm install):

```
sudo kubeadm join 10.0.0.5:6443 --token <t> \
  --discovery-token-ca-cert-hash sha256:<h>
```

On control plane:

```
k get nodes
# NAME      STATUS    ROLES    AGE   VERSION
# cp-1      Ready     control-plane  5m    v1.32.0
# w-1      Ready     <none>      1m    v1.32.0
```

To set worker role label cosmetically:

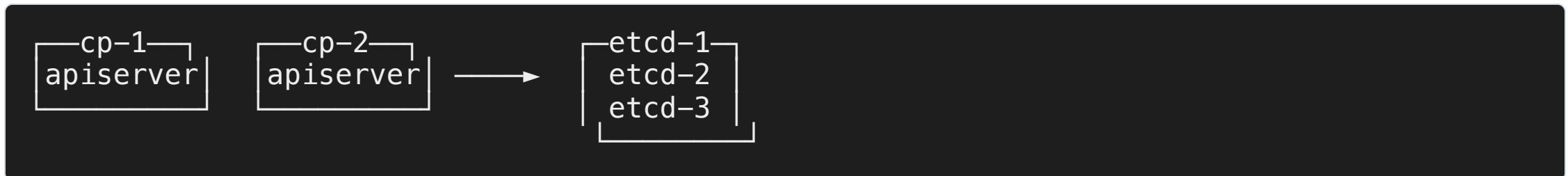
```
k label node w-1 node-role.kubernetes.io/worker=
```

Control plane HA — two topologies

Stacked etcd (kubeadm default):



External etcd:



Stacked: simpler, fewer machines.

External: stronger blast-radius isolation, harder ops, recommended for very large clusters.

HA init flow

```
# init the first control plane with --control-plane-endpoint pointing at the LB
sudo kubeadm init \
  --control-plane-endpoint=cluster.example.com:6443 \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16

# join the other CPs
sudo kubeadm join cluster.example.com:6443 \
  --token <t> --discovery-token-ca-cert-hash sha256:<h> \
  --control-plane --certificate-key <k>
```

`--upload-certs` puts the cluster's certs into a kubeadm-managed Secret so other CPs can pull them. The `--certificate-key` is printed once at init time; print again with `kubeadm init phase upload-certs --upload-certs`.

kubeadm upgrade flow (memorize this)

```
# CONTROL PLANE (one CP at a time in HA)
k drain <cp> --ignore-daemonsets
sudo apt-get update
sudo apt-get install -y kubeadm=1.32.x-*
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.32.x
# (on additional CPs: `kubeadm upgrade node` instead of `apply`)

sudo apt-get install -y kubelet=1.32.x-* kubectl=1.32.x-*
sudo systemctl daemon-reload && sudo systemctl restart kubelet
k uncordon <cp>

# WORKERS (one at a time)
k drain <w> --ignore-daemonsets
# on worker:
sudo apt-get install -y kubeadm=1.32.x-*
sudo kubeadm upgrade node
sudo apt-get install -y kubelet=1.32.x-* kubectl=1.32.x-*
sudo systemctl daemon-reload && sudo systemctl restart kubelet
k uncordon <w>
```

TLS certificate management

kubeadm generates these certs under `/etc/kubernetes/pki/`:

Cert	What it's for	Default expiry
<code>apiserver.crt</code>	apiserver TLS	1 year
<code>apiserver-kubelet-client.crt</code>	apiserver → kubelet	1 year
<code>front-proxy-client.crt</code>	aggregation layer	1 year
<code>etcd/server.crt</code> , <code>peer.crt</code> , <code>healthcheck-client.crt</code>	etcd	1 year
<code>ca.crt</code> , <code>etcd/ca.crt</code> , <code>front-proxy-ca.crt</code>	CAs	10 years
Kubeconfigs (<code>admin.conf</code> , <code>controller-manager.conf</code> , <code>scheduler.conf</code> , <code>kubelet.conf</code>)	client certs embedded	1 year

Implication: if you don't `kubeadm upgrade` for a year, certs expire and the cluster falls over.

Check + renew certs

```
sudo kubeadm certs check-expiration
```

```
# CERTIFICATE          EXPIRES          RESIDUAL TIME    ...
# admin.conf           Mar 02, 2026 14:13 UTC 300d             ...
# apiserver            Mar 02, 2026 14:13 UTC 300d             ...
# ...
```

Renew all at once:

```
sudo kubeadm certs renew all
sudo systemctl restart kubelet           # reload kube-apiserver static pod
```

Renew one:

```
sudo kubeadm certs renew apiserver
```

`kubeadm upgrade` also renews certs as a side-effect. That's the recommended path.

Kubelet client cert auto-rotation

kubelet rotates its own client cert if enabled:

```
# /var/lib/kubelet/config.yaml
rotateCertificates: true
serverTLSBootstrap: true
```

The kubelet renews via the CSR API. Pending CSRs need approval (auto-approved by the `csrapproving` controller if its RBAC is in place).

```
k get csr
k certificate approve <csr-name>
```

Rotate the cluster CA (rare, painful)

If the CA itself is compromised, you must:

1. Generate a new CA
2. Re-issue every certificate signed by it
3. Distribute the new CA to every kubelet, every kubeconfig, every controller
4. Update every TLS Secret / Webhook config

kubeadm has limited support for this. In practice: rebuild the cluster. Plan for **before** you hit year 10.

Lab 7 — kubeadm

→ `trainees/day4/labs/lab7-kubeadm.md`

90 min.

- Bring up 1 cp + 1 worker on Hetzner VMs (trainer provisioned via OpenTofu)
- Install Calico, get to Ready
- `kubeadm certs check-expiration` to inspect the PKI
- (Stretch) Do a `kubeadm upgrade plan` dry-run

etcd — the database

Holds **all cluster state**. If etcd is gone, the cluster is gone.

- Runs as a **static pod** on each control-plane node (kubeadm default)
- TLS-mutual auth via certs in `/etc/kubernetes/pki/etcd/`
- Listens `:2379` (clients), `:2380` (peer-to-peer)
- Uses **Raft** consensus — odd-numbered membership (3 or 5)
- Quorum: `floor(N/2) + 1`. Lose more than that → cluster halts writes.

Backup is non-negotiable. Schedule a snapshot at least every 6h on production.

etcd snapshot save

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/snap.db \  
--endpoints=https://127.0.0.1:2379 \  
--cacert=/etc/kubernetes/pki/etcd/ca.crt \  
--cert=/etc/kubernetes/pki/etcd/server.crt \  
--key=/etc/kubernetes/pki/etcd/server.key
```

```
ETCDCTL_API=3 etcdctl snapshot status /tmp/snap.db -w table
```

```
# +-----+-----+-----+-----+  
# |  HASH  | REVISION | TOTAL KEYS | TOTAL SIZE |  
# +-----+-----+-----+-----+  
# | 8c1a... |    12345 |     2317   |    4.2 MB   |  
# +-----+-----+-----+-----+
```

Memorize the four flag names: `--endpoints --cacert --cert --key`. Practice until you can type them without docs.

etcd restore flow

```
# 1. Stop control plane: move static manifests out
sudo mv /etc/kubernetes/manifests /etc/kubernetes/manifests.bak

# 2. Wait for kubelet to tear down the static pods
sudo crictl ps                # should be empty for cp components

# 3. Restore to a NEW data dir (don't overwrite the live one)
sudo ETCDCTL_API=3 etcdctl snapshot restore /tmp/snap.db \
  --data-dir=/var/lib/etcd-restore

# 4. Swap data dirs
sudo mv /var/lib/etcd /var/lib/etcd.old
sudo mv /var/lib/etcd-restore /var/lib/etcd

# 5. Restart control plane
sudo mv /etc/kubernetes/manifests.bak /etc/kubernetes/manifests

# 6. Verify
sudo crictl ps                # apiserver + etcd back
k get nodes
```

Lab 8 — etcd backup & restore

→ `trainees/day4/labs/lab8-etcd-backup.md`

45 min.

- Take a snapshot
- Create some objects (a Deployment, a CM)
- Delete them
- Restore from the snapshot
- Verify the deleted objects are back

This is on every CKA exam in some form. Practice until it's muscle memory.

CRDs and operators — the extension model

Kubernetes is extensible *at runtime*. The two main mechanisms:

1. **CRD (Custom Resource Definition)** — register a new Kind with the apiserver. Now you can `kubectl get <yourkind>`.
2. **Operator** — a controller that watches your CRD and reconciles real-world state to match. Same pattern as built-in controllers.

You don't write operators on the CKA. You should be able to **recognize** them, install them, debug them.

A CRD example

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata: { name: backups.example.com }
spec:
  group: example.com
  scope: Namespaced
  names:
    plural: backups
    singular: backup
    kind: Backup
    shortNames: [bk]
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                schedule: { type: string }
                retention: { type: integer }
              required: [schedule]
```

Custom Resource (CR)

```
apiVersion: example.com/v1
kind: Backup
metadata: { name: nightly, namespace: prod }
spec:
  schedule: "0 2 * * *"
  retention: 14
```

A CR is just a YAML object that conforms to the CRD's schema. Stored in etcd like every other object.

A CR **alone** does nothing — same as an Ingress alone. You need a controller to act on it.

What an operator looks like

Most operators ship as a Deployment in some namespace + RBAC + the CRD.

```
k apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.yaml
k get crd | grep cert-manager
# certificaterequests.cert-manager.io
# certificates.cert-manager.io
# clusterissuers.cert-manager.io
# ...
k -n cert-manager get pods
# cert-manager-*      Running    ← the controller
# cert-manager-webhook-*  Running
# cert-manager-cainjector-*
```

The controller watches Certificate CRs and creates real Secrets containing TLS material. Same reconcile pattern.

Debug an operator

If your CR sits there doing nothing:

1. `k describe <cr>` — look for status conditions and events
2. `k -n <operator-ns> logs <operator-pod>` — what is the controller saying
3. `k get apiservice` — if it's an aggregated API (rare), is it `Available: True`?
4. `k get crd | grep <group>` — CRD installed? Versions match what you're applying?
5. RBAC — operator SA must have permissions on every Kind it touches

When you'll see operators on the exam

CRDs are *conceptual* on CKA — they may ask you to:

- Identify what a CRD is from its YAML
- Apply a provided CR after a CRD is installed
- Read events/status from a CR
- Understand why "nothing is happening" (operator pod down)

You won't write a CRD from scratch.

Lab 8b — CRDs & operators

→ `trainees/day4/labs/lab8b-crds.md`

30 min.

- Install cert-manager
- Inspect installed CRDs
- Create a self-signed Issuer + a Certificate
- Watch the controller create the Secret
- Break the operator (delete its Deployment); watch reconciliation stop

RBAC + ServiceAccounts revisited for troubleshooting

The shape of a "permission denied" error:

```
Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:ns:sa"
cannot list resource "pods" in API group "" in the namespace "ns"
```

Read it carefully: who, what verb, what resource, what API group, what namespace.

Then:

```
k auth can-i list pods -n ns --as=system:serviceaccount:ns:sa
k -n ns get rolebindings -o wide
k -n ns describe rolebinding <name>
```

Troubleshooting playbook (the order)

```
1. kubectl get <kind>           ← status?
2. kubectl describe <kind>      ← events at the bottom?
3. kubectl logs <pod>          ← --previous if crashed; -c for multi-container
4. Node level                   ← journalctl -u kubelet
                                crictl ps -a / crictl logs
5. Component level             ← /etc/kubernetes/manifests/*
                                kube-apiserver logs (via container)
6. Network                     ← endpoints, NetworkPolicy, DNS
                                nslookup from a pod
7. Auth                        ← kubectl auth can-i; RBAC
```

This is your decision tree. Memorize it. Run it top-down on every broken thing.

Logging — what kubectl gives you

```
k logs <pod>
k logs <pod> -c <container>           # multi-container
k logs <pod> --previous                 # last crash
k logs -f <pod>                        # stream
k logs -l app=web --max-log-requests=20 --tail=50

# from a node, raw:
sudo journalctl -u kubelet -n 200 --no-pager
sudo ls /var/log/pods/<ns>_<pod>_<uid>/<container>/
sudo crictl ps -a
sudo crictl logs <id>
```

Logs come from container stdout/stderr → kubelet → `/var/log/pods/...` → kubectl streams.

If a container writes to a **file**, kubelet doesn't see it. Use a sidecar to tail to stdout.

Cluster-wide logging architecture

Pattern	How	When
Node-level DaemonSet	Fluent Bit / Vector / Promtail tails <code>/var/log/containers/*.log</code> on every node, ships to central store	90% of clusters
Sidecar	Extra container in each pod streams app log files	App can't write to stdout; legacy apps
App pushes directly	App library sends to backend	Loses node context; ad-hoc

Production backends: Loki, Elastic/OpenSearch, Datadog, CloudWatch, GCP Logging.

Out of CKA exam scope, but you should know the **shape** of the pipeline.

Metrics — metrics-server

Required for `kubectl top` and HPA.

```
k apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# on kind, edit metrics-server deployment, add to container args:
# - --kubelet-insecure-tls

k top nodes
k top pods -A
```

`metrics-server` serves the `metrics.k8s.io` API. It is **not** Prometheus — it only keeps the *most recent* metrics in memory.

For real monitoring: **kube-prometheus-stack** Helm chart (Prometheus + Grafana + Alertmanager + node-exporter). Out of CKA scope.

The common failure table

Symptom	First command	Then
Node <code>NotReady</code>	<code>journalctl -u kubelet -n 100</code> on that node	container runtime? swap? cert expired?
Pod <code>Pending</code>	<code>k describe pod</code> events	scheduling: insufficient cpu, taints, affinity
Pod <code>ImagePullBackOff</code>	<code>k describe pod</code>	typo, private registry → <code>imagePullSecrets</code>
Pod <code>CrashLoopBackOff</code>	<code>k logs --previous</code>	exit code; entrypoint; failing probe
Pod <code>CreateContainerConfigError</code>	<code>k describe pod</code>	referenced CM/Secret doesn't exist
Service unreachable	<code>k get endpoints <svc></code>	empty? selector wrong / pods not Ready
DNS broken	<code>k -n kube-system get pods -l k8s-app=kube-dns</code>	CoreDNS logs; Corefile
API hangs / no response	<code>apiserver static pod: etcd</code>	<code>/etc/kubernetes/manifests/</code> ;

Static pod recovery

If apiserver / scheduler / controller-manager / etcd is broken:

```
ls /etc/kubernetes/manifests/  
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml
```

Edit the file — kubelet sees the change in ~10s and restarts the pod.

If you broke it badly:

```
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/  
# kubelet tears it down  
sudo crictl ps                                # confirm gone  
# fix the file, move it back  
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/
```

This is the **only** way to "restart" a control plane static pod. There's no `systemctl restart kube-apiserver`.

Network troubleshooting flow

Service unreachable

- check endpoints (empty? → selector / readiness / pods)
- from a pod: nslookup <svc> (DNS? → CoreDNS)
- from a pod: curl <svc-ip> (kube-proxy? iptables? CNI?)
- check NetworkPolicy on source + dest
- check kube-proxy logs on the receiving node

Spawn a debug pod with networking tools:

```
k run dbg --rm -it --image=nicolaka/netshoot -- bash
> dig +search +noall +answer web
> curl -v http://web
> tcpdump -ni any port 80
> ss -tlpn
```

Lab 9 — Troubleshooting

→ `trainees/day4/labs/lab9-troubleshooting.md`

45 min. Trainer assigns ONE of four scenarios at random:

- **A. Broken kubelet** on a node (service config wrong)
- **B. Broken control plane** (corrupt static pod manifest)
- **C. Broken DNS** (CoreDNS Corefile loop / wrong upstream)
- **D. Broken app** (probe, image, RBAC, NetworkPolicy — pick one)

Diagnose, fix, prove. Write the root cause in `/tmp/lab9-rca.txt` (good habit for the exam).

Mock Exam

→ `trainees/day4/labs/mock-exam.md`

75 minutes. 13 questions. No notes. No Google. No AI.

Allowed: `kubernetes.io/docs`, `helm.sh/docs`, `kubernetes.io/blog`.

Set a hard timer. Trainer reviews after time is up.

Exam-day strategy — the rules I'd give myself

- **Skim ALL questions first** (5 min). Mark easy / medium / hard. Solve easy first.
- Every question has a **weight (%)**. A 10% question is worth 2.5× a 4% question.
- **Set context + namespace on every question**. Forgetting costs you the whole question.
- **Stuck > 8 min?** Flag and move on. Come back at the end.
- Use `k explain --recursive <kind>` instead of digging through docs pages.
- Use `$do` to generate; edit; apply. Don't type Pod YAML from scratch.
- `k edit` is dangerous — easy to mess up YAML in vim with no undo to last apply.
- Always verify with the question's success criteria *before* moving on (a service "must work" means `curl` it).

Day-of logistics

- **Eat first.** No food during the exam.
- **Water in a clear glass.** Proctor needs to see through it.
- **Clear desk.** Proctor will scan the room before starting.
- **One monitor.** Disconnect external displays.
- **Chrome only.** Firefox/Safari are not supported by PSI.
- **Bathroom break** is permitted once (newer rule; check the current Candidate Handbook).
- **ID:** government ID with Latin characters (passport is safest).

After the exam

- Result via email in ~24 hours
- **If you fail:** 1 free retake within 12 months. Book it **2 weeks out**, do killer.sh once more in between.
- **If you pass:** digital badge on Credly, valid **2 years**.

Where to go next

- **CKS** (Certified Kubernetes Security Specialist) — needs active CKA, 2-year window
- **KCSA** — security associate, no prereq
- **CKAD** — developer counterpart (some overlap, less ops)
- Real-world skill stack: Helm chart authoring, GitOps (Argo CD / Flux), service mesh (Istio/Linkerd), observability (Prometheus + Grafana + Loki)
- Operator authoring (Kubebuilder, Operator SDK) — natural Day 5 if this becomes a habit

Wrap-up

Thank you for the four days. Three things to do this week:

1. **killer.sh session 1** within 7 days (it will feel impossibly hard — that's the point)
2. **Schedule the exam** (book a date — accountability matters)
3. **Re-run the mock exam** from this course on a fresh kind cluster

Questions?